

Mobile Application Development

A practical study reference with foundational concepts, examples, and common review questions.

1. Platforms and Architecture

Android and iOS have distinct SDKs, lifecycle rules, UI conventions, permissions, and distribution. Native development uses each platform's primary tools; cross-platform frameworks share code but still need native integration. Choose based on audience, device features, team skills, performance, and maintenance.

Separate presentation, business/domain logic, and data access. Keep rendering apart from business rules so behavior can be tested. Verify current platform requirements and store policies before release.

2. Lifecycle, UI, Accessibility

Apps can move to background, be suspended, or be recreated after process death. Persist important state instead of assuming a process lives forever. Handle rotation, low memory, navigation, and interruptions.

Adapt to screen size, text scaling, safe areas, and input methods. Accessibility requires semantic labels, contrast, logical focus, screen-reader support, and usable touch targets. Test using platform accessibility tools and real devices.

3. Network, Storage, and Privacy

Network operations are asynchronous and may time out or fail. Use TLS and bounded timeouts. Retry carefully; non-idempotent operations need an idempotency strategy. Provide meaningful offline, loading, and error states.

Store only necessary data. Use platform secure storage for credentials and keys, request permissions only when needed, and gracefully handle denial or revocation. Define consent, retention, deletion, and synchronization conflict behavior.

4. Testing and Common Questions

Unit tests cover logic, integration tests cover boundaries, and UI tests exercise flows. Test multiple devices, OS versions, offline conditions, slow networks, and denied permissions. Profile startup, memory, battery, networking, and rendering before optimizing.

Native apps directly use platform APIs; cross-platform apps share more code but may need platform-specific refinement. Keep network work off the UI thread. Deep links are input and must not bypass authorization. Offline-first design keeps useful local behavior and defines synchronization rules.